

Les Bases Python
Opérateurs et Affectation
- Affectation = : Assigne une valeur (à droite) à une variable (à gauche). - Addition + : Somme - Soustraction - : Différence - Multiplication * : Produit. - Division / : Division classique (renvoie toujours un 'float'). - Division entière // : Renvoie uniquement la partie entière du quotient. - Modulo % : Renvoie le reste de la division entière. - Puissance ** : Élève à l'exposant.
Affectation Combinée
Modifie une variable en utilisant sa propre valeur. On a +=, -=, *=, /=, C'est équivalent à x = x + y.
Opérateurs de Comparaison
Renvoient True ou False :
== : Égal à != : Différent de. < : Strictement inférieur à. > : Strictement supérieur à. <= : Inférieur ou égal à. >= : Supérieur ou égal à.
Types de Données Fondamentaux
On peut voir le type de une variable avec la commande type(variable)
- Nombre entier int : Chiffre entier - Nombre décimal float : Utilisation du point - Valeur de vérité logique bool : True ou False - Chaîne de caractères str : Délimité par "..." ou '...' - Liste list : Délimité par [...] - Dictionnaire dict : Délimité par {...}
Opérateurs Logiques
and : Toutes les conditions doivent être vraies.

- or : Au moins une des conditions doit être vraie. - not : Inverse le résultat logique
Chaînes de Caractères (str)
En Python, une chaîne de caractères ne peut pas être modifiée sur place. Toutes les méthodes ci-dessous ne changent pas la chaîne d'origine, elles <i>renvoient une nouvelle chaîne</i> modifiée (qu'il faut souvent stocker dans une variable).
Opérations Fondamentales et Longueur
- len(chaîne) : Longueur,Fonction native (non-méthode) renvoyant le nombre total de caractères (int). - chaîne1 + chaîne2 : Concaténation,Fusionne deux chaînes bout à bout. - chaîne * entier : Répétition,Répète la chaîne le nombre de fois spécifié.
Indexation et Slicing
- chaîne[index] : Extrait le caractère à la position précise. (Un index négatif comme -1 cible le dernier caractère). - chaîne[début:fin] : Extrait de l'index début (inclus) jusqu'à l'index fin (exclu). - chaîne[début:fin:pas] : Extrait du début à la fin en sautant des caractères selon le pas.
Mettre un vide à debut va prendre par default 0, pour la fin c'est le dernier element et pour pas c'est 1.
Modification de la Casse
- chaîne.upper() : Convertit tous les caractères en majuscules. - chaîne.lower() : Convertit tous les caractères en minuscules.
Liste
Dictionnaire
Fonctions
Structure Générale:
<pre>def nom_fo(p_obl, p_opt=val): # Corps de la fonction [instructions] return [valeur_a_renvoyer]</pre>

- def : Mot-clé déclarant la création de la fonction. (...): Déclare les paramètres. :: Obligatoire - Indentation : Délimite formellement ce qui appartient (ou non) à la fonction. - return : Renvoie le résultat au programme principal. Son exécution termine immédiatement la fonction. Sans return explicite, la fonction renvoie None.
Fonctions Récursives
Une fonction récursive est une fonction qui <i>s'appelle elle-même</i> à l'intérieur de son propre bloc d'instructions.
<pre>def f_r([parametres]): # 1. Condition d'arrêt if [condition_de_fin_atteinte]: return [valeur_finale] # 2. Appel Récursif else: # Modification possible [new_p] = [modification]</pre>
La fonction s'invoque elle-même return [operation] + f_r([new_p])
Les composants indispensables sont :
- Condition d'arrêt : Évalue si le problème est suffisamment réduit pour être résolu directement sans nouvel appel - Modification d'état : Les arguments passés à l'appel suivant doivent systématiquement se rapprocher de la condition d'arrêt - Le mot-clé return : Permet de faire remonter le résultat d'un appel au précédent lors du "dépilage".
Structures Conditionnelles
Structure Générale
<pre>if [condition_1]: [instructions_1] elif [condition_2]: # Bloc exécuté après [instructions_2] else: [instructions_sinon]</pre>

On a :
- Si if : : Point d'entrée obligatoire - Sinon si elif : : Optionnel et répétable - Sinoncelse : : Optionnel et unique
Les règles d'exécution sont : Ordre de lecture , Exclusivité et Syntaxe stricte
Les Boucles
Boucle while (Tant que)
Conçue pour itérer tant qu'une condition reste vraie. Le nombre de répétitions n'est pas connu à l'avance .
<pre>while [condition]: # Bloc exécuté tant que True [instructions] [mise_à_jour_de_la_condition]</pre>
Il est impératif de modifier une variable intervenant dans la condition à l'intérieur du bloc, sinon la boucle tournera à l'infini!
Boucle for (Pour chaque)
Conçue pour itérer sur une séquence connue (liste, chaîne de caractères, dictionnaire, etc.). La boucle s'arrête automatiquement à la fin de la séquence.
<pre>for [var_temporaire] in [séquence]: # Bloc exécuté une fois par élément [instructions_avec_var_temporaire]</pre>
À chaque tour, la var_temporaire prend automatiquement la valeur de l'élément suivant dans la séquence.
L'outil range()
Génère une séquence de nombres entiers. Idéal pour répéter une action un nombre de fois précis.
- range(fin) : De 0 jusqu'à fin - 1. - range(début, fin) : De début jusqu'à fin - 1. - range(début, fin, pas) : De début jusqu'à fin - 1, en avançant selon le pas.
Mots-clés de Contrôle
Ces mots-clés s'utilisent à l'intérieur du bloc indenté des boucles for ou while (souvent combinés avec un if).

- break : Arrête l'itération actuelle et sort définitivement de la boucle entière. - pass : Ne fait rien (bien visuellement)
Package
Math
Numpy
random
Ce module doit être importé via import random. Il sert à générer des nombres pseudo-aléatoires. La comande qui doit être toujours mise (pour la reproductibilité) est random.seed([valeur]).
Les comandes bien à savoir sont :
- random.random() : Renvoie un nombre décimal (<i>float</i>) aléatoire compris dans l'intervalle [0, 1[. - random.uniform([min], [max]) : Renvoie un nombre décimal (<i>float</i>) aléatoire compris entre [min, max] - random.randint([min], [max]) : Renvoie un nombre entier (<i>int</i>) aléatoire compris entre [min, max] - random.choice([séquence]) : Renvoie un seul élément choisi au hasard dans la séquence non vide. - random.sample([séquence], k=[nombre]) : Renvoie une liste de éléments de la séquence tirés au hasard sans remise.
Matplotlib
Le module est toujours importé via import matplotlib.pyplot as plt.
Types de Graphiques Principaux (Initialisation)
Ces fonctions tracent les données sur la figure active. Elles acceptent toutes un argument optionnel label=[texte] utile pour générer la légende.
- plt.plot([x], [y]) : Courbe reliant les points (Graphique linéaire). - plt.scatter([x], [y]) : Nuage de points (Points indépendants). - plt.bar([x], [hauteurs]) : Diagramme en barres verticales.

- `plt.hist([données], bins=[entier])` : Histogramme (Répartition des données dans des intervalles/bacs)
- `plt.pie([valeurs], labels=[étiquettes])` : Diagramme circulaire (Camembert)

Habillage et Annotations

Ces fonctions s'appliquent au graphique actif pour lui ajouter du contexte et des informations:

- `plt.title([texte])` : Ajoute un titre principal au-dessus du graphique.
- `plt.xlabel([texte])`
`plt.ylabel([texte])` : Ajoute un titre à l'axe des abscisses
- `plt.xlim([min], [max])`
`plt.ylim([min], [max])` : Force les limites d'affichage des axes
- `plt.legend()` : Affiche la légende (nécessite d'avoir renseigné l'argument `label=`)

Final

Obligatoire: Calcule le rendu final et ouvre la fenêtre d'affichage du graphique.

```
plt.show()
```